

COMPTE-RENDU L'ANALYSE DE L'ARTICLE DE ROBOTIQUE

Robin Didier & Lubin Caillol

1 Introduction, motivations

L'article *A General Framework for Managing Multiple Tasks in Highly Redundant Robotic Systems* (Siciliano & Slotine, 1991), s'attaque à l'un des défis majeurs de la robotique avancée : comment permettre à un robot complexe de **réaliser plusieurs tâches en même temps sans perdre le contrôle**.

Pour comprendre ce défi, rappelons la base. Un robot est défini par ses coordonnées articulaires

$$q = \begin{pmatrix} q_1 \\ \vdots \\ q_n \end{pmatrix} \in \mathbb{R}^n.$$

Si le i^e joint est rotatif, alors q_i est un angle. Si le joint est prismatique (vérin, allongement par exemple), alors q_i est une distance.

La position de l'outil au bout du robot, lui, est défini par un vecteur $x \in \mathbb{R}^m$. Par exemple cela peut être une description de la pose complète en 3D (6 degrés de liberté) :

$$x = \begin{pmatrix} p_x \\ p_y \\ p_z \\ \phi \\ \theta \\ \psi \end{pmatrix}.$$

La relation entre ces angles et la position de l'outil x est donnée par la cinématique directe :

$$x = f(q).$$

Pour contrôler le robot en mouvement, on dérive la relation spatiale par rapport au temps :

$$\dot{x} = J \dot{q} \tag{*}$$

où $J = \frac{\partial f}{\partial q}$ est la matrice Jacobienne (de dimension $m \times n$).

On parle de **redondance** lorsque $n > m$. Le robot a en quelque sorte trop de moteurs pour la tâche demandée. **On suppose par la suite traiter le cas de robots redondants.**

Mettons que l'on se fixe une position souhaitée x . On veut trouver q réalisant ce dernier. Nous, on va considérer des cas de robot redondant, donc J n'est pas carrée. Nous utilisons la matrice pseudo-inverse de Moore-Penrose : $J^\# = J^T (J J^T)^{-1}$. Cette matrice possède la propriété très utile pour le calcul :

$$J J^\# J = J.$$

La solution générale à notre problème s'écrit :

$$\dot{q} = J^\# \dot{x} + (I - J^\# J) z$$

Évidemment, comme le robot est redondant, il existe une infinité de solutions q . Cette flexibilité est donnée par le terme $I - J^\# J$ qui est le projecteur sur le noyau de J . Il filtre n'importe quel **vecteur de vitesse arbitraire** z de sorte que son effet sur la vitesse de l'outil (sur $\dot{x}$) soit strictement nul.

Démonstration.

Déjà, $p = I - J^\# J$ est un projecteur car $p \circ p = (I - J^\# J)(I - J^\# J) = I - 2J^\# J + J^\# J J^\# J$.
 Comme $J J^\# J = J$, on a $p \circ p = I - 2J^\# J + J^\# J = p$.
 C'est donc un projecteur.

On a aussi la propriété suivante :

$$J(I - J^\# J) = J - J = 0,$$

ce qui implique que

$$\forall z \in \mathbb{R}^m, (I - J^\# J)z \in \text{Ker}(J).$$

On conclut en remarquant que $\dot{x} = J\dot{q}$. Notre z est donc un vecteur de vitesses articulaires $\dot{q}$ arbitraire qui sera projeté de façon à ne pas faire bouger notre effecteur.

Analyse des approches antérieures : Le problème de l'inversion en accélération

Pour comprendre la rupture apportée par Siciliano et Slotine, il faut analyser comment les travaux précédents tentaient de gérer la dynamique. Ces méthodes cherchaient à résoudre le problème au niveau des accélérations (Computed-Torque Control) en dérivant (*) :

$$\ddot{x} = J\ddot{q} + \dot{J}\dot{q}.$$

En isolant l'accélération articulaire $\ddot{q}$, on obtient une solution de la forme :

$$\ddot{q} = J^\#(\ddot{x} - \dot{J}\dot{q}) + (I - J^\# J)z.$$

Pourquoi est-ce instable ?

Le problème fondamental ici est l'absence de retour sur la vitesse (velocity feedback) dans le noyau.

- **Dérive numérique** : Puisque l'accélération est la double dérivée de la position, toute erreur de mesure ou de discrétisation dans $\ddot{q}$ s'intègre deux fois. Cela cause une dérive non contrôlée de la vitesse articulaire $\dot{q}$.
- **Instabilité Interne** : Dans le noyau de J , aucune contrainte ne stabilise les vitesses. Le robot peut développer des mouvements internes (self-motion) de grande amplitude qui divergent vers l'infini, alors que l'effecteur semble suivre la trajectoire.

Le changement de paradigme

Plutôt que d'inverser l'accélération, on va voir que l'approche de Siciliano et Slotine repose sur une stratégie de **contrôle au niveau vitesse** et de la **dérivation analytique**.

Ainsi, $\ddot{q}$ devient une *conséquence* d'une vitesse $\dot{q}$ déjà bornée et contrôlée. Cette inversion de priorité (contrôler la vitesse pour déduire l'accélération, et non l'inverse) est la clé de la stabilité du système proposé.

2 Le résultat majeur : le framework de priorité des tâches

Le cœur de l'article de Siciliano et Slotine réside dans la formulation d'un algorithme récursif capable de gérer un nombre arbitraire de tâches tout en respectant une hiérarchie de priorité stricte.

2.1 Définition de la hiérarchie et Jacobienne Augmentée

Supposons que nous ayons k tâches à accomplir simultanément. La tâche 1 possède la priorité la plus élevée, et la tâche k la plus basse. Pour chaque tâche i , on définit :

- $x^i \in \mathbb{R}^{m_i}$: le vecteur de la tâche i .
- $J^i = \frac{\partial f_i}{\partial q}$: la Jacobienne associée à cette tâche.

Concrètement : la tâche numéro i doit s'exécuter en respectant les contraintes imposées par les tâches $1, \dots, i-1$. On définit alors la **Jacobienne Augmentée** J_A^{i-1} comme l'empilement des Jacobiennes des tâches prioritaires :

$$J_A^{i-1} = \begin{pmatrix} J^1 \\ J^2 \\ \vdots \\ J^{i-1} \end{pmatrix} \in \mathbb{R}^{(\sum m_j) \times n}$$

L'objectif est de trouver une vitesse articulaire $\dot{q}$ qui satisfait $\dot{x}_i = J_i \dot{q}$ sans modifier les valeurs de $\dot{x}^1, \dots, \dot{x}^{i-1}$ déjà calculées.

2.2 La solution récursive en vitesse

Le résultat fondamental est la loi de mise à jour récursive de la vitesse articulaire.

Théorème : solution récursive de priorité

Soit $\dot{q}^{i-1}$ la solution satisfaisant les $i-1$ premières tâches. La formule de récurrence permettant de calculer $\dot{q}^i$ satisfaisant les tâches $1, \dots, i$ est la suivante.

$$\begin{aligned} \dot{q}^i &= \dot{q}^{i-1} + \bar{J}^i \# (\dot{x}^i - J^i \dot{q}^{i-1}) \\ \dot{q}^1 &= J^1 \# \dot{x}^1 \end{aligned}$$

où :

- $\dot{x}^i$ est la vitesse désirée pour la tâche i .
- $J^i \dot{q}^{i-1}$ est la vitesse de la tâche i déjà induite par les tâches plus prioritaires.
- $\bar{J}^i = J^i P_A^{i-1}$ est la Jacobienne projetée.
- $P_A^{i-1} = I - (J_A^{i-1})^\# J_A^{i-1}$ est le projecteur sur le noyau de la Jacobienne augmentée.

La philosophie de l'algorithme : un principe de non-interférence

Si l'on prend un peu de recul sur cette équation mathématique, sa philosophie est en réalité très intuitive. L'algorithme procède par accumulation stricte :

1. **La base de travail** : On part de $\dot{q}^{i-1}$. C'est la commande de vitesse articulaire qui garantit l'exécution parfaite des $i-1$ tâches les plus prioritaires. C'est notre acquis, on ne doit plus le détruire.
2. **L'effort résiduel** : On évalue ce qu'il reste à faire pour la tâche i . C'est le terme d'erreur $(\dot{x}^i - J^i \dot{q}^{i-1})$. Il représente la vitesse manquante pour accomplir la tâche i , sachant que le robot est déjà en train de bouger à la vitesse $\dot{q}^{i-1}$.

3. **La modification sous contrainte :** Pour combler cette erreur, on ajoute un terme de modification des vitesses articulaires. Mais attention, ce terme n'est pas calculé avec la Jacobienne classique J^i , il est calculé avec la Jacobienne projetée $\bar{J}^i = J^i P_A^{i-1}$. Intuitivement, si J^i est ce que la tâche i veut faire, $\bar{J}^i$ est ce que la tâche i peut faire en fonction de sa priorité.

Ainsi, par construction, la matrice pseudo-inverse $\bar{J}^{i\#}$ génère des vecteurs de vitesse qui appartiennent *exclusivement* à l'image du projecteur P_A^{i-1} . Autrement dit, le terme de modification que l'on ajoute vit intégralement dans le **noyau de la Jacobienne des tâches supérieures**.

On sollicite donc les moteurs pour réaliser la tâche i , mais cette sollicitation est mathématiquement filtrée pour que son impact sur les tâches 1 à $i - 1$ soit **strictement nul**. Le robot utilise sa redondance (ses degrés de liberté restants) pour s'accommoder de la nouvelle tâche sans jamais interférer avec ce qui est plus important.

2.3 Démonstration du framework récursif

Pour une tâche i , nous cherchons à déterminer la vitesse articulaire $\dot{q}^i$ satisfaisant la tâche i sans perturber les tâches $1, \dots, i - 1$.

Démonstration.

Soit $\dot{q}^{i-1}$ la vitesse articulaire satisfaisant les $i - 1$ premières tâches. Nous cherchons la solution sous la forme :

$$\dot{q}^i = \dot{q}^{i-1} + P_A^{i-1} \Delta \dot{q}^i$$

où P_A^{i-1} est le projecteur sur l'espace nul des tâches prioritaires. Par définition, $P_A^{i-1} \Delta \dot{q}^i$ n'affecte pas les tâches $1, \dots, i - 1$.

1. Satisfaction de la tâche i

On impose $J^i \dot{q}^i = \dot{x}^i$. En substituant notre forme de $\dot{q}^i$:

$$J^i (\dot{q}^{i-1} + P_A^{i-1} \Delta \dot{q}^i) = \dot{x}^i,$$

donc

$$J^i P_A^{i-1} \Delta \dot{q}^i = \dot{x}^i - J^i \dot{q}^{i-1}.$$

2. Résolution pour $\Delta \dot{q}^i$

Posons $\bar{J}^i = J^i P_A^{i-1}$. Il s'agit de la Jacobienne projetée (ou Jacobienne de la tâche i vue dans le noyau des précédentes). La solution de norme minimale est :

$$\Delta \dot{q}^i = \bar{J}^{i\#} (\dot{x}^i - J^i \dot{q}^{i-1}).$$

3. Mise à jour du projecteur

Le nouveau projecteur P_A^i doit annuler non seulement l'espace nul des tâches $1 \dots i - 1$, mais aussi la nouvelle contrainte imposée par la tâche i . On peut démontrer que :

$$P_A^i = P_A^{i-1} - \bar{J}^{i\#} \bar{J}^i.$$

Cette relation est une généralisation du projecteur sur l'espace nul $I - J^\# J$ vu dans la partie I : où on partait de tout l'espace (I) et soustrayait la partie occupée par la contrainte ($J^\# J$). Cette fois, on part de P_A^{i-1} .

Cette relation de récurrence permet de calculer P_A^i à partir de P_A^{i-1} sans avoir à recalculer explicitement la matrice Jacobienne augmentée globale J_A^i .

D'où le résultat.

2.4 Stabilité par dérivation analytique

La grande force de ce *framework* est d'assurer une *stabilité interne*. Concrètement, l'article montre que pour contrôler le robot (en accélération), il faut utiliser les $\dot{q}_1, \dots, \dot{q}_i$ afin d'obtenir les $\ddot{q}_1, \dots, \ddot{q}_i$ de la façon (à nouveau) récursive suivante :

$$\begin{aligned}\ddot{q}^i &= \ddot{q}^{i-1} + \bar{J}^{i\#}(\ddot{x}^i - \dot{J}^i \dot{q}^{i-1} - J^i \ddot{q}^{i-1}) + \dot{\bar{J}}^{i\#}(\dot{x}^i - J^i \dot{q}^{i-1}) \\ \ddot{q}^1 &= J^{1\#} \ddot{x}^1 + \dot{J}^{1\#} \dot{x}^1\end{aligned}$$

Cette formulation complexe garantit qu'il n'y a **aucune dérive numérique de la vitesse** dans l'espace nul. Toutes les composantes du mouvement sont rigoureusement sous contrôle. Si l'effecteur est à l'arrêt, les vitesses internes des articulations sont mathématiquement contraintes de rester stables, éliminant ainsi les mouvements internes divergents (self-motions) qui posaient problème dans les méthodes précédentes.

2.5 La flexibilité de la hiérarchie

Un autre point fondamental soulevé par l'article est la remise en question du statut de la tâche principale. Historiquement en robotique, la tâche 1 est presque toujours le suivi de trajectoire de l'effecteur (la main du robot). Les tâches secondaires (évitement d'obstacle, optimisation d'énergie) s'exécutent dans l'espace nul de cette tâche principale.

Cependant, Siciliano et Slotine soulignent que ce paradigme est limitant, particulièrement dans des environnements encombrés (comme on le verra avec le robot serpent).

- Si la tâche de position de l'effecteur est tâche 1, et l'évitement d'obstacle est tâche 2, le robot **n'évitera l'obstacle que si cela ne perturbe pas sa trajectoire**. S'il n'a pas le choix, il percutera l'obstacle pour garder son effecteur sur la ligne.
- La solution : inverser la hiérarchie. On définit l'évitement d'obstacle comme tâche 1, et la trajectoire de l'effecteur comme tâche 2. Le robot va alors se déformer pour contourner l'obstacle à tout prix, et utilisera ses degrés de liberté restants pour ramener son effecteur le plus près possible de la cible désirée. La sécurité prime sur la précision.

On va maintenant illustrer ces concepts par un exemple calculatoire simple, mais d'abord il nous faut faire quelques précisions sur la notion de *tâche*.

Toute contrainte s'exprimant sous forme différentielle, notée $\dot{x} = J\dot{q}$, peut être intégrée naturellement dans le processus récursif. Cette flexibilité repose sur deux interprétations de la Jacobienne J :

- **Tâches cartésiennes** : Si la tâche concerne l'effecteur, $x = f(q)$ représente la cinématique directe. La Jacobienne J correspond alors à la *Jacobienne géométrique* classique, obtenue par dérivation de $f(q)$ par rapport aux articulations.
- **Tâches articulaires (contraintes internes)** : Si la tâche consiste à imposer une valeur à une articulation (ex : bloquer q_i), le mouvement est défini directement dans l'espace articulaire. Dans ce cas, la fonction de transfert f est l'identité. La matrice J devient alors une *matrice de sélection* (composée de 0 et de 1), isolant le degré de liberté visé.

Cette uniformisation est la clé de la puissance de cet algorithme : le projecteur P_A^i traite avec la même rigueur mathématique une contrainte géométrique complexe et une simple consigne de blocage moteur. L'exemple suivant illustre cette dualité.

Exemple d'illustration : Application stricte du formalisme

Imaginons un bras robotique plan avec 3 articulations, dont les positions sont définies par le vecteur

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix} \in \mathbb{R}^3.$$

Le robot est redondant pour une tâche de positionnement de son effecteur dans un plan 2D. Définissons les tâches.

- **Tâche 1** (prioritaire). Bloquer la première articulation. Conformément à ce qui a été expliqué dans le paragraphe précédent, on a pour un tel type de tâche $x^1 = q_1$: la tâche ne concerne que la première composante de q . La Jacobienne associée est une matrice de sélection : $J^1 = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}$.
- **Tâche 2** (secondaire). Déplacer l'effecteur à une vitesse cartésienne désirée $\dot{x}^2 = \begin{pmatrix} \dot{p}_x \\ \dot{p}_y \end{pmatrix} \in \mathbb{R}^2$.

On note la Jacobienne géométrique de l'effecteur $J^2 = \begin{pmatrix} J_{11}^2 & J_{12}^2 & J_{13}^2 \\ J_{21}^2 & J_{22}^2 & J_{23}^2 \end{pmatrix}$.

Étape 1 : Résolution de la tâche 1

On cherche la vitesse articulaire $\dot{q}^1$ permettant de satisfaire la première tâche.

$$\dot{q}^1 = J^{1\#} \dot{x}^1.$$

Puisque $\dot{x}^1 = 0$, on obtient :

$$\dot{q}^1 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Étape 2 : Jacobienne Augmentée et Projecteur P_A^1

Pour protéger la tâche 1, on définit la Jacobienne augmentée de l'étape 1. Puisqu'il n'y a qu'une seule tâche prioritaire, elle est simplement :

$$J_A^1 = J^1.$$

Le projecteur sur l'espace nul de cette Jacobienne augmentée se calcule par $P_A^1 = I - J_A^{1\#} J_A^1$:

$$P_A^1 = I - \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \end{pmatrix} = I - \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Étape 3 : La Jacobienne Projetée $\bar{J}^2$

La tâche 2 doit être projetée dans l'espace nul de la tâche 1. On calcule la Jacobienne projetée $\bar{J}^2 = J^2 P_A^1$:

$$\bar{J}^2 = \begin{pmatrix} J_{11}^2 & J_{12}^2 & J_{13}^2 \\ J_{21}^2 & J_{22}^2 & J_{23}^2 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & J_{12}^2 & J_{13}^2 \\ 0 & J_{22}^2 & J_{23}^2 \end{pmatrix}.$$

Étape 4 : Solution finale $\dot{q}^2$

On applique la relation de récurrence du framework pour trouver la commande finale :

$$\dot{q}^2 = \dot{q}^1 + \bar{J}^{2\#} (\dot{x}^2 - J^2 \dot{q}^1).$$

En remplaçant $\dot{q}^1$ par le vecteur nul, l'équation se simplifie :

$$\dot{q}^2 = \bar{J}^{2\#} \dot{x}^2$$

Conclusion : La matrice $\bar{J}^2$ possédant une première colonne de zéros, sa pseudo-inverse $\bar{J}^{2\#}$ possédera une première ligne de zéros. Ainsi, le calcul de $\dot{q}^2$ ne produira *aucune* vitesse sur l'articulation q_1 . Le framework garantit formellement que **la tâche 2 sera accomplie en utilisant exclusivement q_2 et q_3 , respectant strictement la tâche 1.**

3 Limites, singularités et implémentation

L'algorithme récursif de Siciliano et Slotine, bien que mathématiquement élégant, se heurte aux limites physiques et numériques inhérentes à la robotique. Dans leur article, les auteurs identifient précisément ces limites et proposent des pistes d'implémentation.

3.1 Analyse des singularités

Les auteurs distinguent deux types de blocages pouvant survenir lors de l'exécution des tâches :

- **La singularité de tâche.** Elle survient lorsque la Jacobienne brute J^i n'est plus de rang plein et est donc singulière (non inversible). Cela correspond à une limite physique du robot par rapport à la tâche i (par exemple, un bras en extension maximale).
- **La singularité algorithmique.** Elle survient lorsque la Jacobienne projetée $\bar{J}^i$ devient singulière, alors même que J^i ne l'est pas. Cela se produit lorsque le mouvement exigé par la tâche i est totalement incompatible avec l'espace nul imposé par les tâches de plus haute priorité.

Notons, d'un point de vue mathématique, que même à l'aide d'une formule de pseudo-inverse, il est numériquement risqué d'en effectuer le calcul lorsque la matrice initiale est singulière ou proche de l'être. Dans notre cas, cela pourrait produire des vitesses très grandes.

Propriété fondamentale.

L'article montre que l'apparition d'une singularité (de tâche ou algorithmique) à l'étape i n'affecte pas le calcul des tâches suivantes ($i + 1, i + 2, \dots$). La dimension de l'espace nul disponible ne diminue pas de façon pathologique, garantissant que le reste de l'algorithme ne s'effondre pas.

Pour éviter que les vitesses articulaires ne divergent à l'approche de ces singularités, l'article préconise de remplacer la pseudo-inverse classique par une méthode des moindres carrés amortis (*damped least-squares*), qui limite les vitesses au prix d'une légère erreur de suivi.

3.2 Efficacité computationnelle

Le calcul de la loi de commande, et particulièrement le passage à l'accélération, exige le calcul de la dérivée de la pseudo-inverse $\dot{\bar{J}}^{i\#}$, une opération numériquement très lourde.

Pour rendre cet algorithme viable en temps réel, les auteurs suggèrent deux optimisations :

1. **Décomposition matricielle :** Au lieu d'inverser brutalement, la matrice J peut être décomposée sous la forme $J = RS$ (où S possède des lignes orthogonales et R est triangulaire). La pseudo-inverse s'obtient alors très efficacement par $J^\# = S^T R^{-1}$.
2. **Filtrage numérique :** Il est possible d'employer des procédures de filtrage pour estimer l'évolution de la matrice sans avoir à dériver analytiquement la pseudo-inverse.

3.3 Illustrations par simulation

On a réalisé un simulateur interactif (Python, PyQtGraph) modélisant un bras manipulateur hyper-redundant à 7 degrés de liberté comme évoqué dans l'article original.

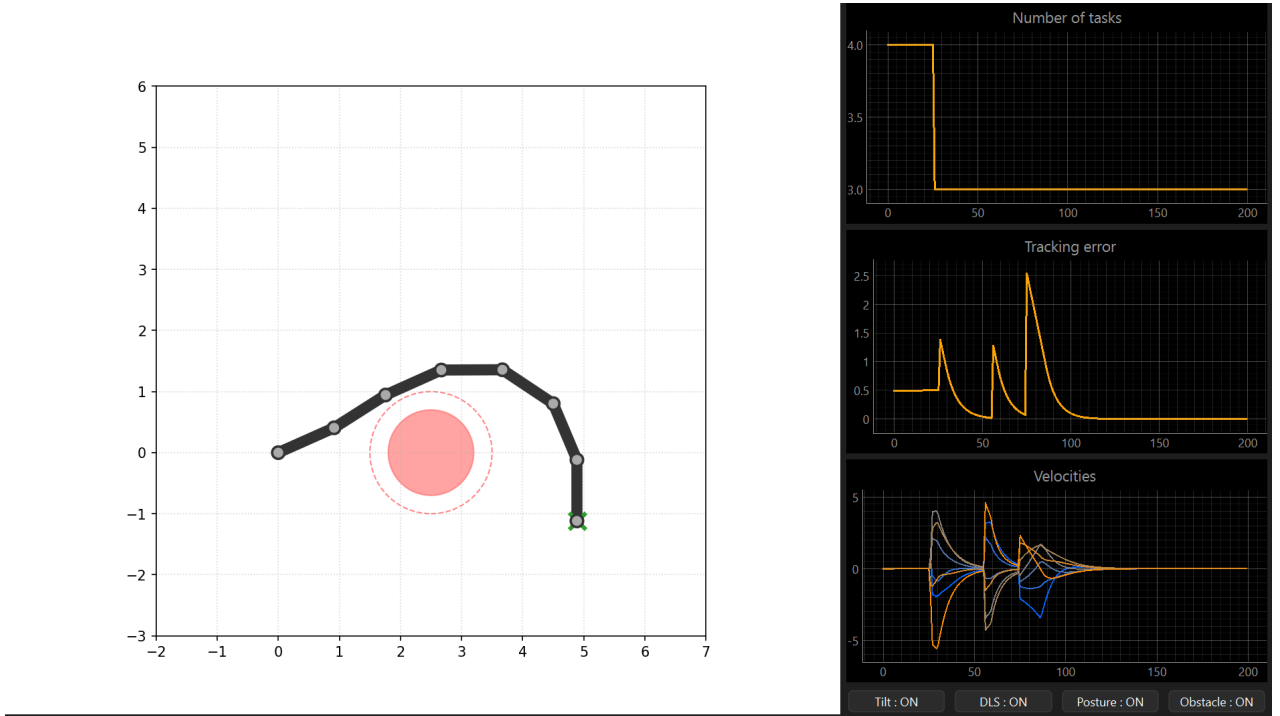


FIGURE 1 – Interface graphique

L'algorithme implémente la loi de commande récursive exacte en vitesse. L'ordre d'ajout des tâches dans notre boucle de calcul définit strictement la hiérarchie suivante :

Priorité	Tâche	Objectif
1	Évitement d'obstacle	Repousse toute articulation menacée hors d'un rayon critique.
2	Orientation	Maintient l'effecteur selon un angle fixe.
3	Suivi	Amène l'effecteur sur une cible (x, y) donnée.
4	Posture	Minimisation du repli des joints : attire q vers le vecteur nul.

L'interface temps réel nous permet de constater visuellement les trois résultats majeurs du formalisme.

Découplage par projection

Scénario : Robot atteignant sa cible (Tâche 3), puis activation de la Posture (Tâche 4).

- **Observation visuelle :** Le robot se reconfigure, repliant ses articulations pour se rapprocher de sa configuration de repos, mais la pointe de l'effecteur ne bouge pas.
- **Observation graphique :** Les courbes des vitesses articulaires oscillent pour effectuer le mouvement, mais la courbe d'*Erreur de tracking* reste à 0.
- **Conclusion :** La tâche de posture s'exécute exclusivement dans le noyau du projecteur P_A^3 . Le principe de non-interférence est empiriquement vérifié.

2. Le renversement de la hiérarchie classique

Scénario : Cible (Tâche 3) placée au centre de l’obstacle, puis activation de l’Évitement (Tâche 1).

- **Observation visuelle** : Dès l’activation, le corps du robot est violemment repoussé de la zone rouge. L’effecteur est contraint de s’éloigner de sa cible.
- **Observation graphique** : L’erreur de tracking s’envole brutalement.
- **Conclusion** : Contrairement aux approches classiques où l’effecteur prime, ici la priorité 1 l’emporte. Il utilise ensuite ses degrés de liberté restants pour minimiser la distance restante à la cible, *sous contrainte* de l’obstacle.

3. La gestion des singularités (damped least-squares)

Scénario : Cible placée hors de portée physique du bras (extension maximale atteinte).

- **Observation graphique (sans DLS)** : La matrice devient numériquement singulière. Les vitesses articulaires ($\dot{q}$) divergent vers l’infini, causant des tremblements.
- **Observation graphique (avec DLS)** : En remplaçant la pseudo-inverse pure par son approximation amortie, les vitesses plafonnent immédiatement à une valeur de sécurité. L’erreur de tracking se stabilise à la distance minimale physiquement réalisable.
- **Conclusion** : Le formalisme de priorité est puissant, mais le traitement numérique de l’inversion conditionne sa viabilité. Le DLS permet un franchissement stable des limites de l’espace de travail.

3.4 Améliorations postérieures du framework

Bien que le formalisme de Siciliano et Slotine ait posé les fondations du contrôle hiérarchique, son recours aux projecteurs et aux pseudo-inverses le restreint fondamentalement au traitement de **contraintes d’égalité** ($\dot{x} = J\dot{q}$).

Dans la pratique, des tâches critiques comme l’évitement d’obstacles ou le respect des butées articulaires s’expriment bien plus naturellement sous forme de **contraintes d’inégalité** ($J\dot{q} \leq b$). Avec les pseudo-inverses classiques, la gestion de ces inégalités nécessite de définir des seuils d’activation. L’activation ou la désactivation soudaine d’une tâche (par exemple, lorsqu’un obstacle entre dans la zone de sécurité) provoque des discontinuités brutales dans les commandes de vitesse, engendrant des à-coups qui peuvent endommager les moteurs.

Pour pallier ce problème, la recherche a opéré un changement de paradigme en abandonnant l’inversion matricielle explicite au profit de la **programmation quadratique hiérarchique (HQP - Hierarchical Quadratic Programming)**.

Un article de référence marquant cette évolution est *Kinematic Control of Redundant Manipulators : Generalizing the Task-Priority Framework to Inequality Task* (Kanoun, Lamiraux, Wieber, 2011). Les auteurs y démontrent comment reformuler la hiérarchie stricte de Siciliano sous la forme d’une cascade de problèmes d’optimisation. Cette approche permet d’intégrer des inégalités de façon native, garantissant des transitions parfaitement lisses et continues lors de l’activation des tâches. C’est aujourd’hui devenu le standard pour le contrôle temps réel de systèmes complexes, comme les robots humanoïdes.